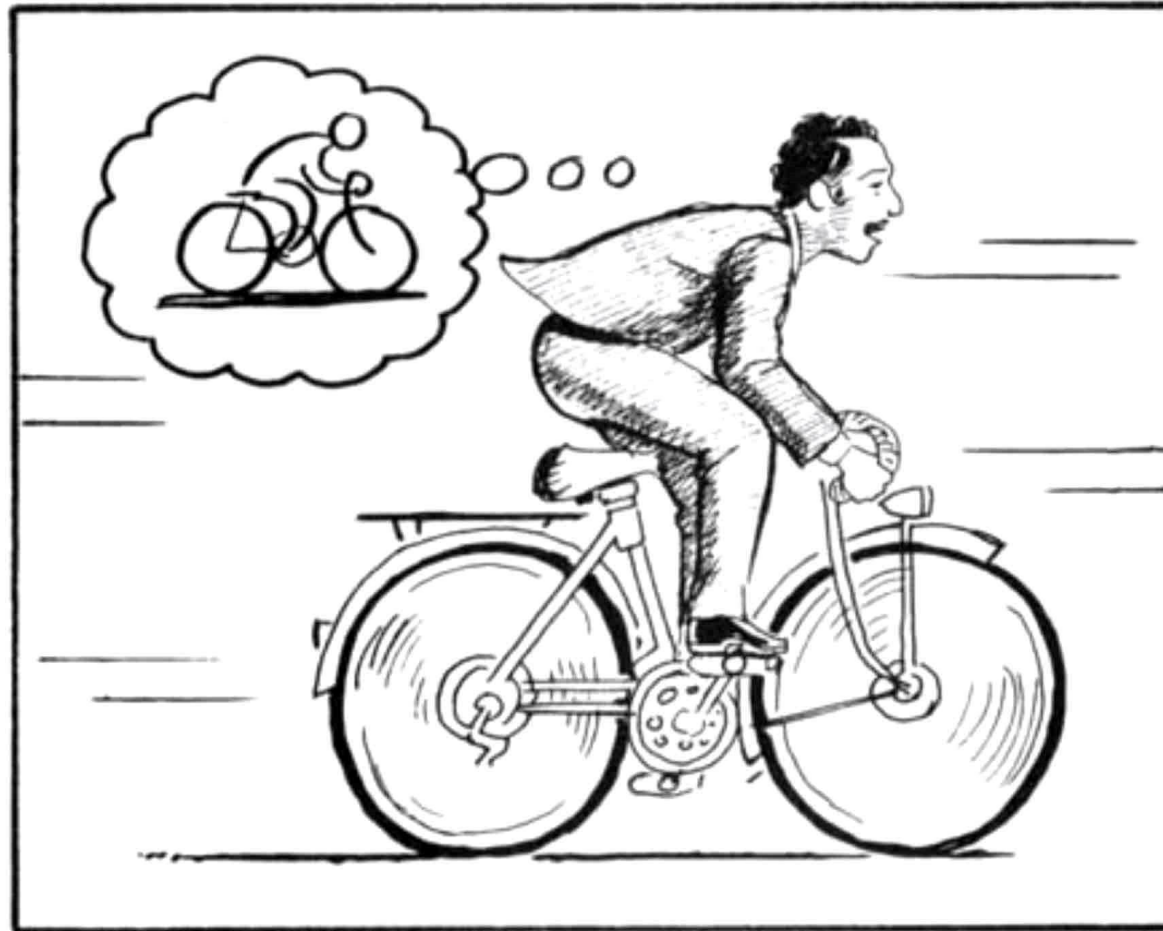
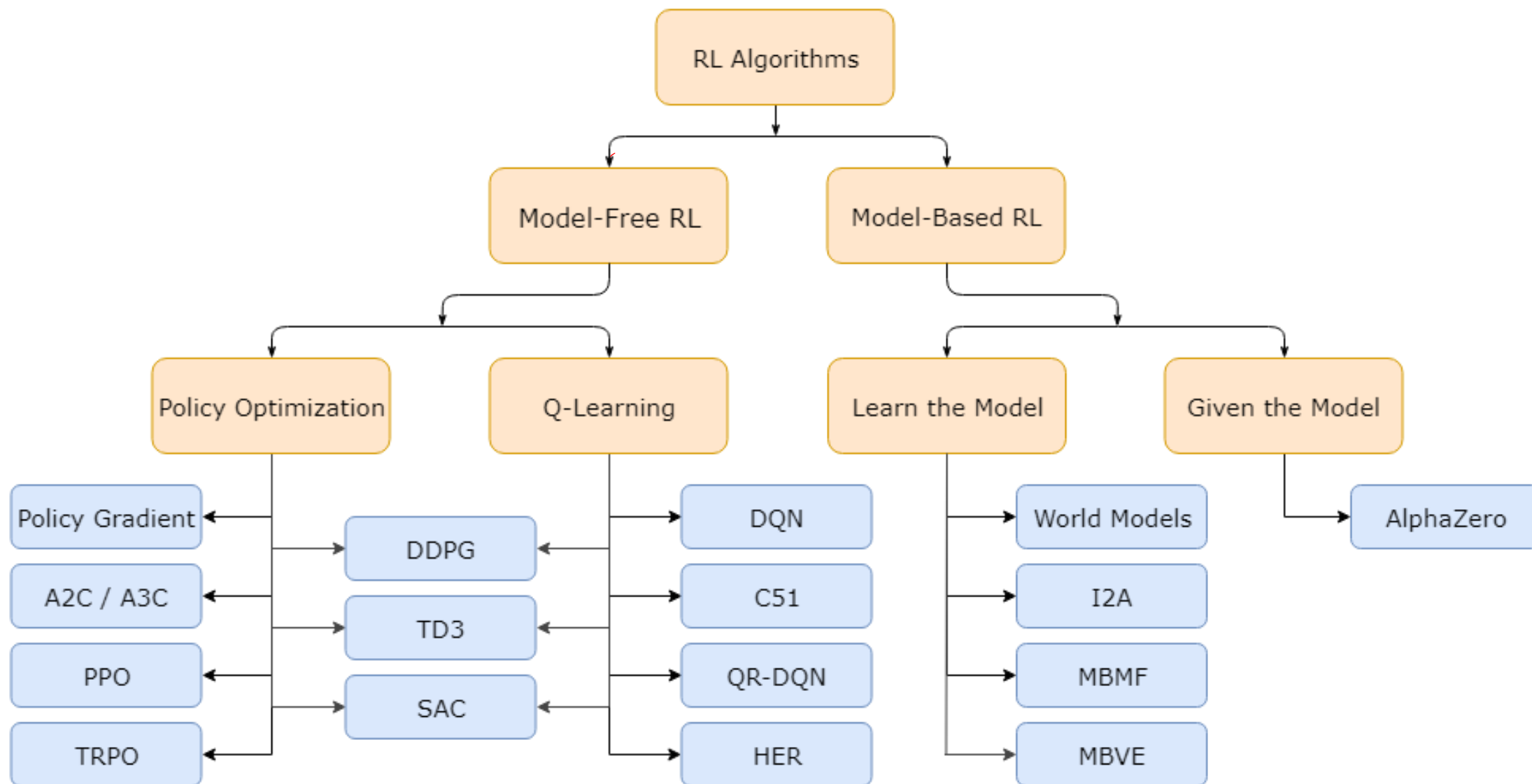


World Models and Model-Based RL

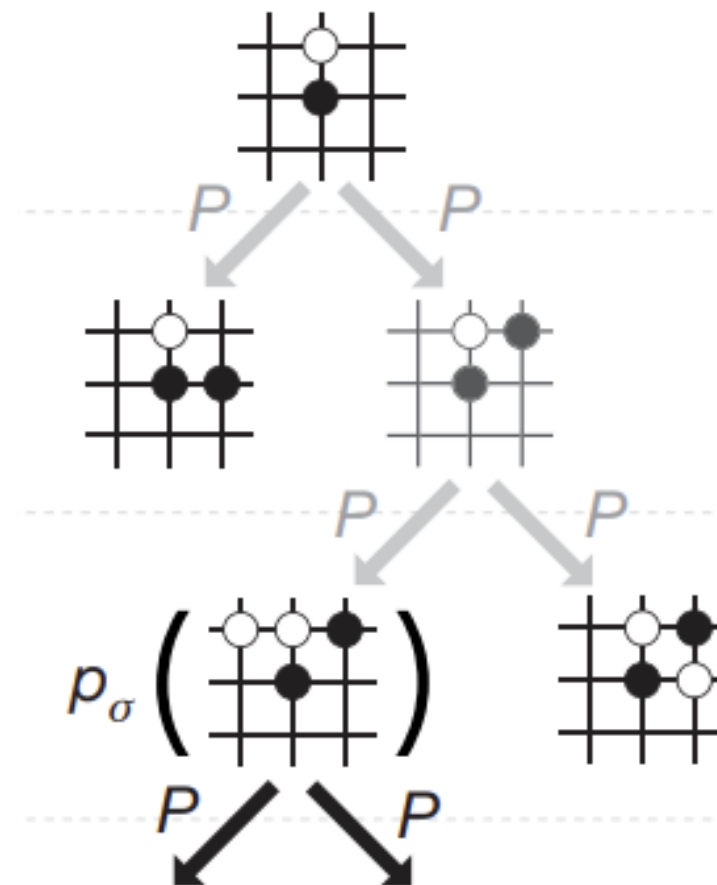
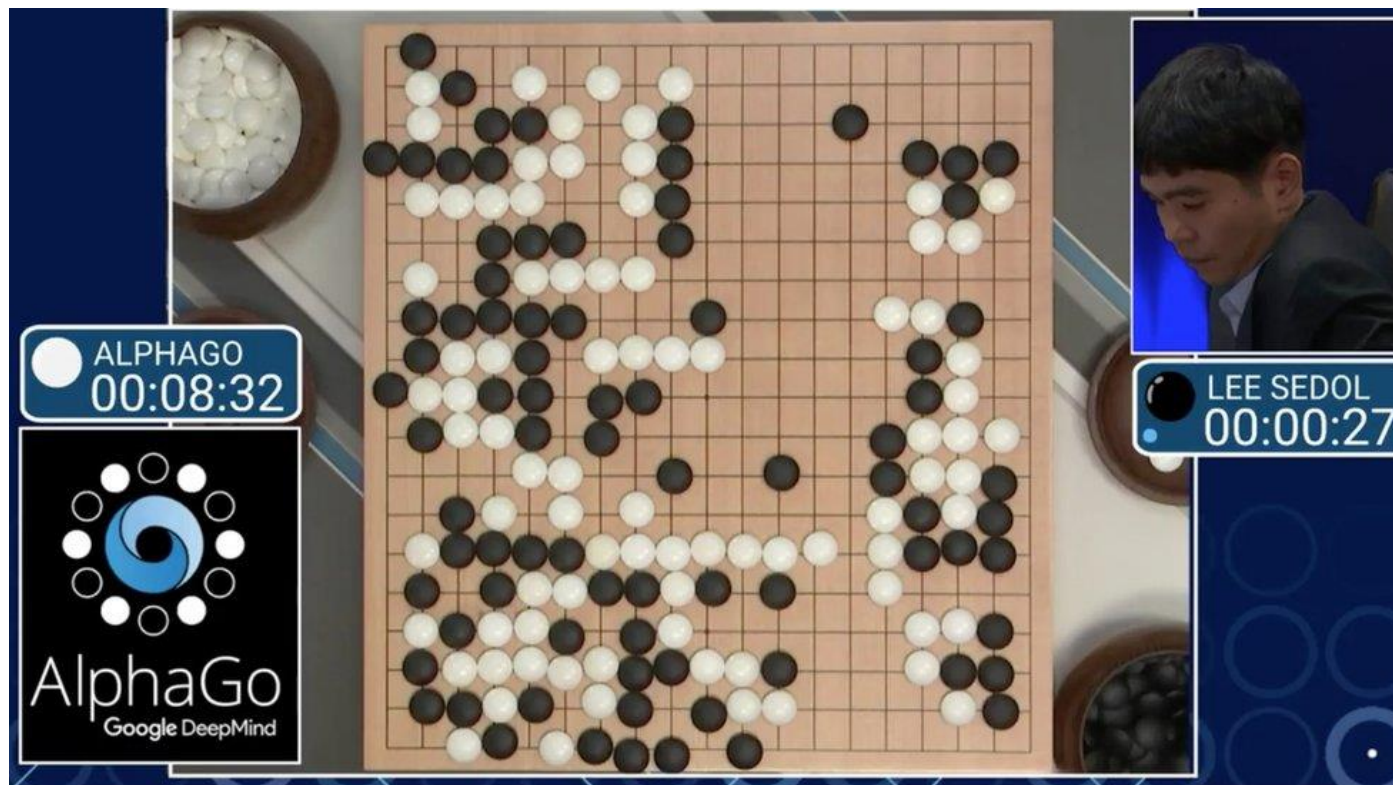


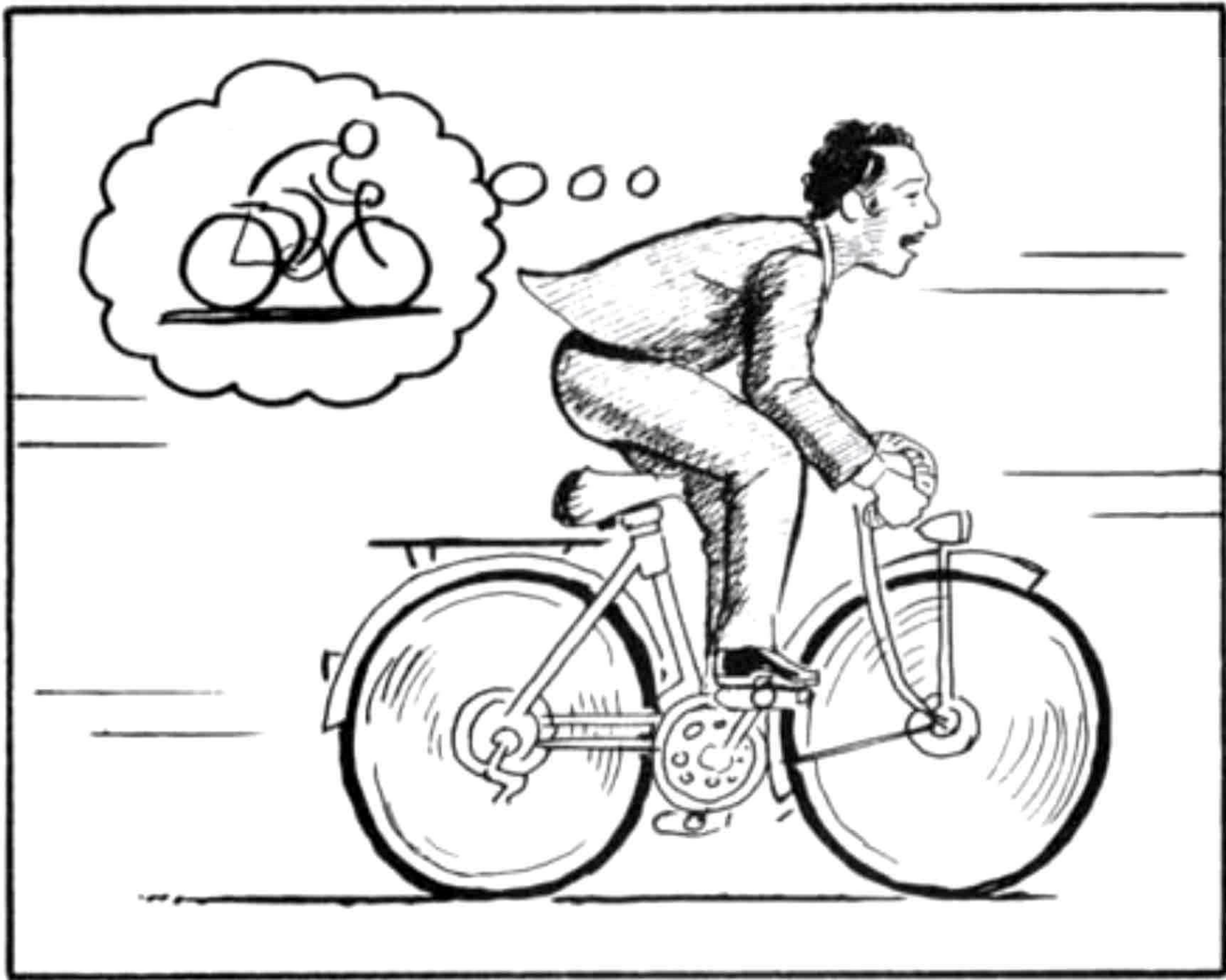
Instructor: Daniel Brown --- University of Utah

Rough Taxonomy of RL Algorithms



Model-Based RL with Known Model

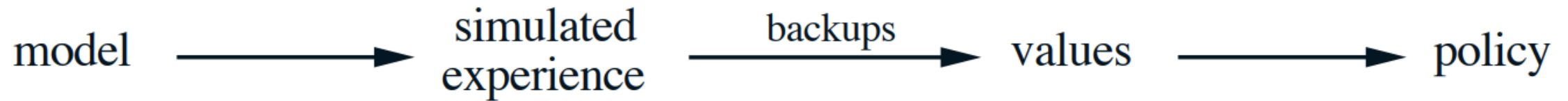




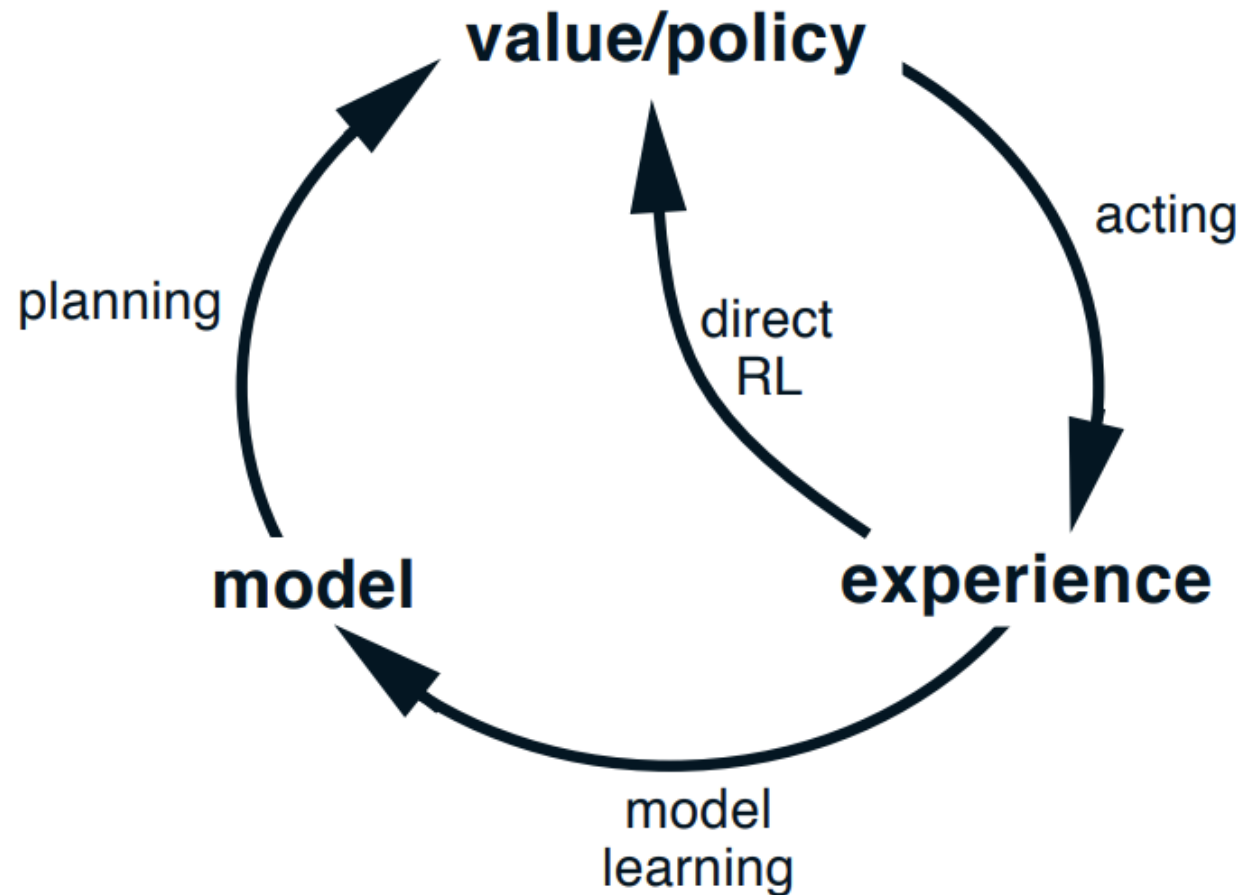
Why use planning over model-free RL?

- More data efficient
- Increased performance just by increasing compute budget for search
- Learned dynamics are task independent

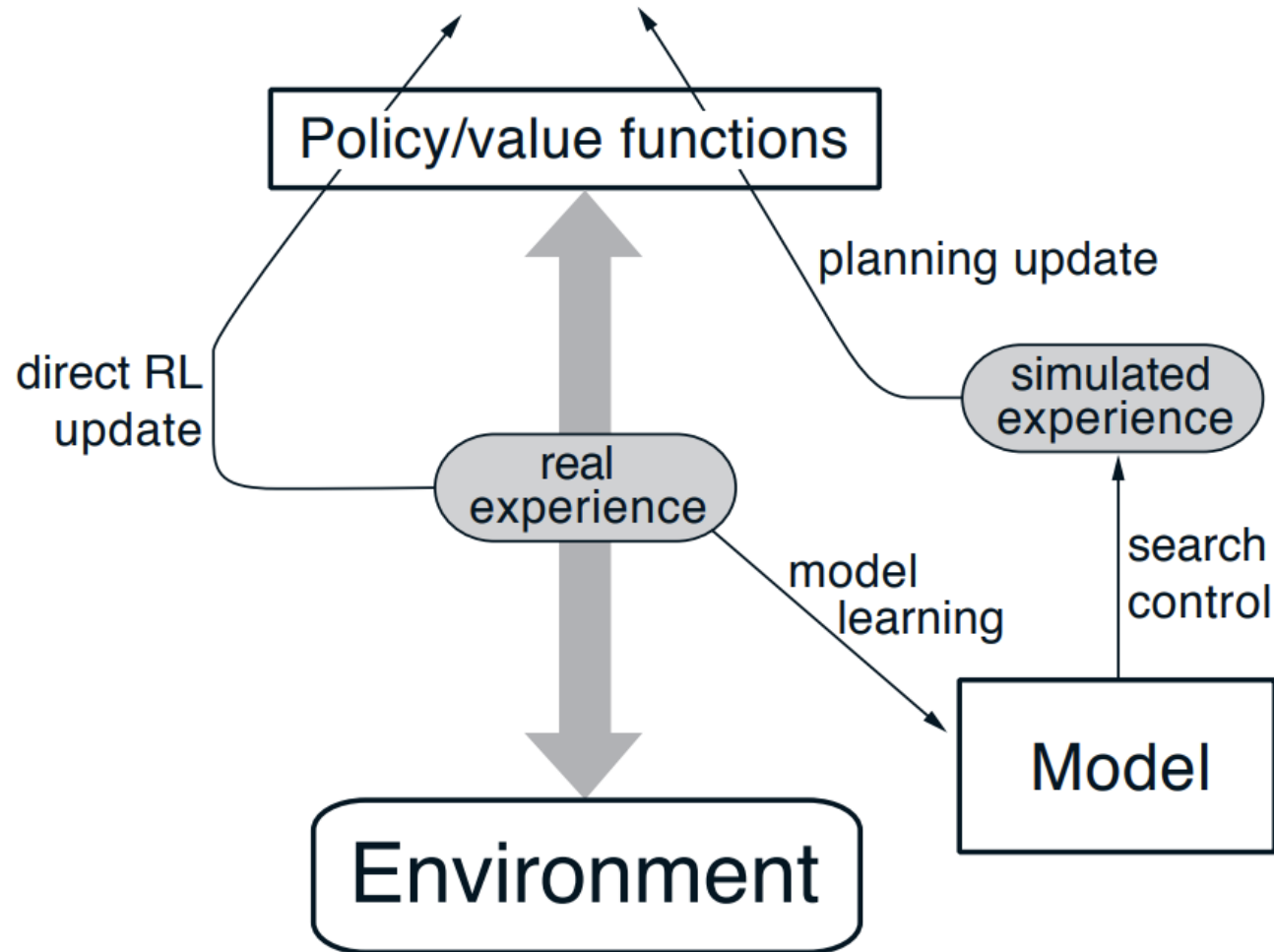
State-Space Planning



Relationships between learning, planning, and acting



Dyna Architecture



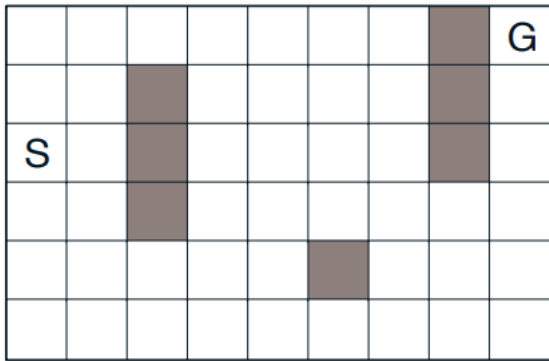
Dyna-Q Algorithm

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

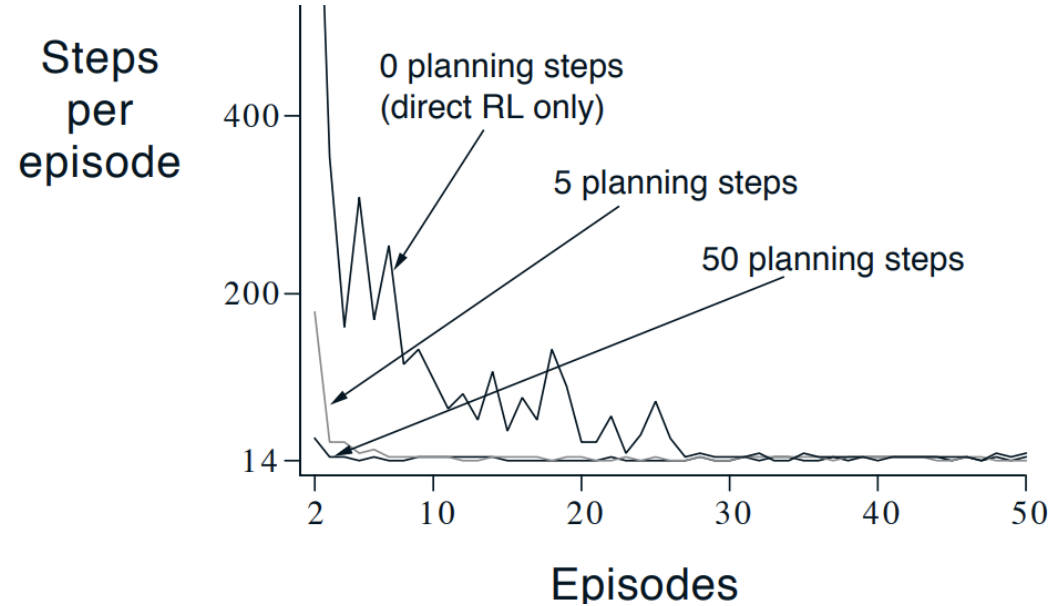
Do forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
- (c) Execute action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

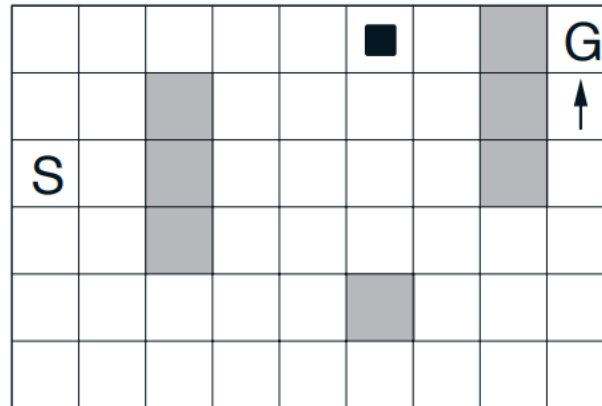
Simple GridWorld Example



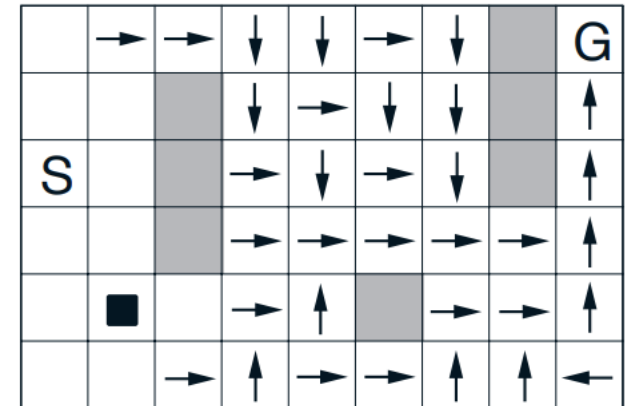
actions



WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



What happens if the model is wrong?

- Optimistic?
- Pessimistic?
- Exploration vs. Exploitation

Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models

Kurtland Chua

Roberto Calandra

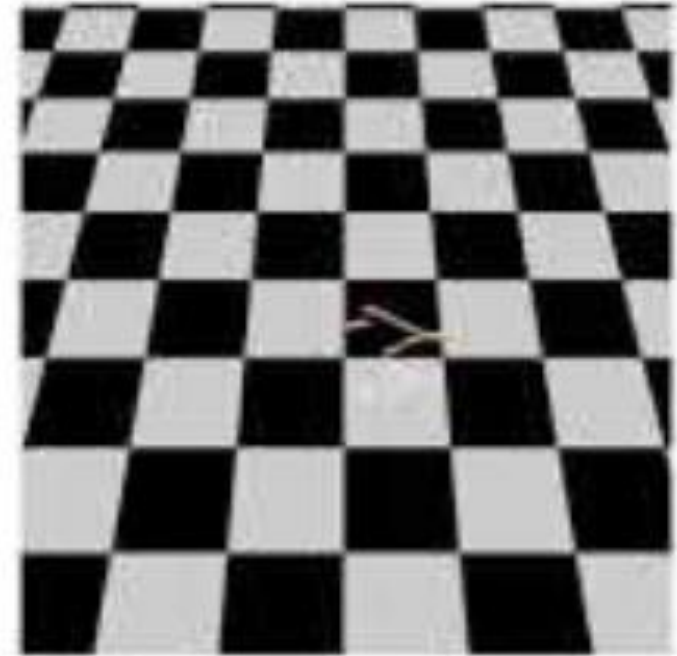
Rowan McAllister

Sergey Levine

Berkeley Artificial Intelligence Research

University of California, Berkeley

`{kchua, roberto.calandra, rmcallister, svlevine}@berkeley.edu`

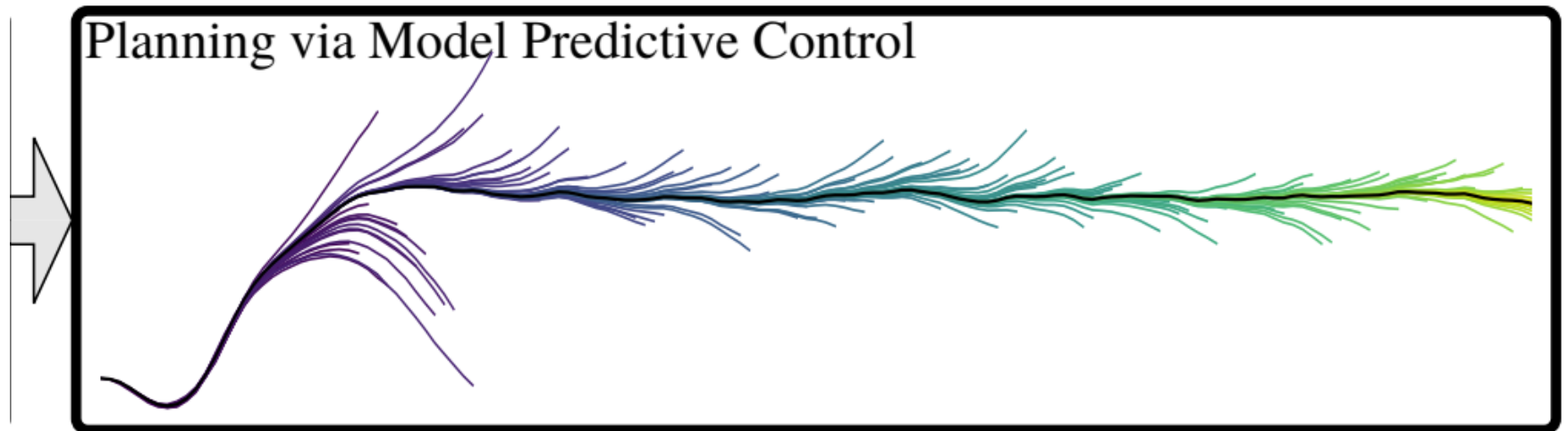
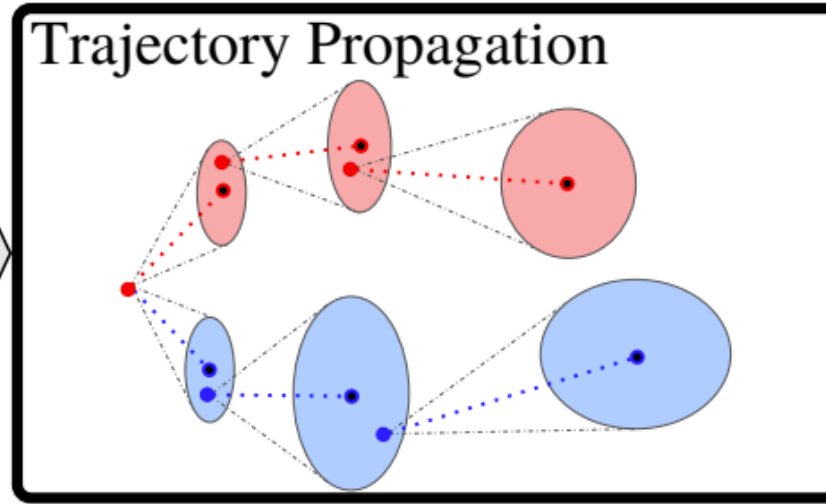
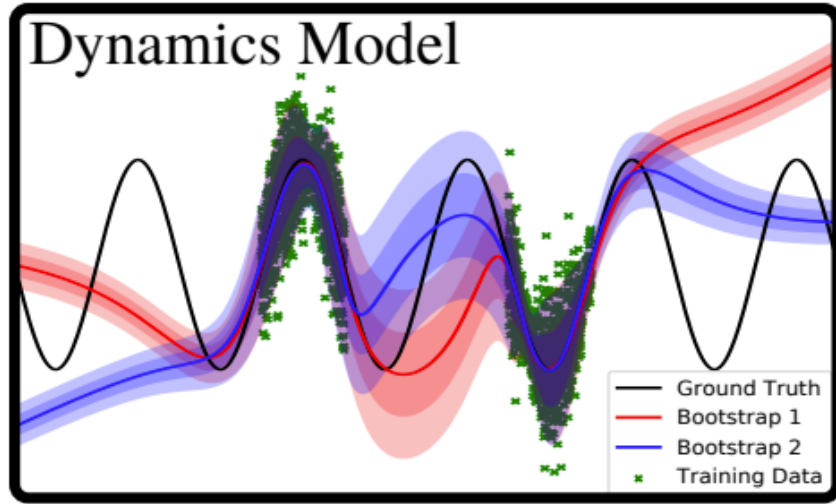


Performance reached with
PE-TS within 100 Trials

Problems

- MF RL methods are expensive
- MB RL methods learn quickly but usually don't reach the end performance of MF methods.
- Neural network models overfit on small datasets

Probabilistic Ensembles with Trajectory Sampling (PETS)



Dealing with Different Types of Uncertainty

- Aleatoric Uncertainty

- The inherent noise in a system

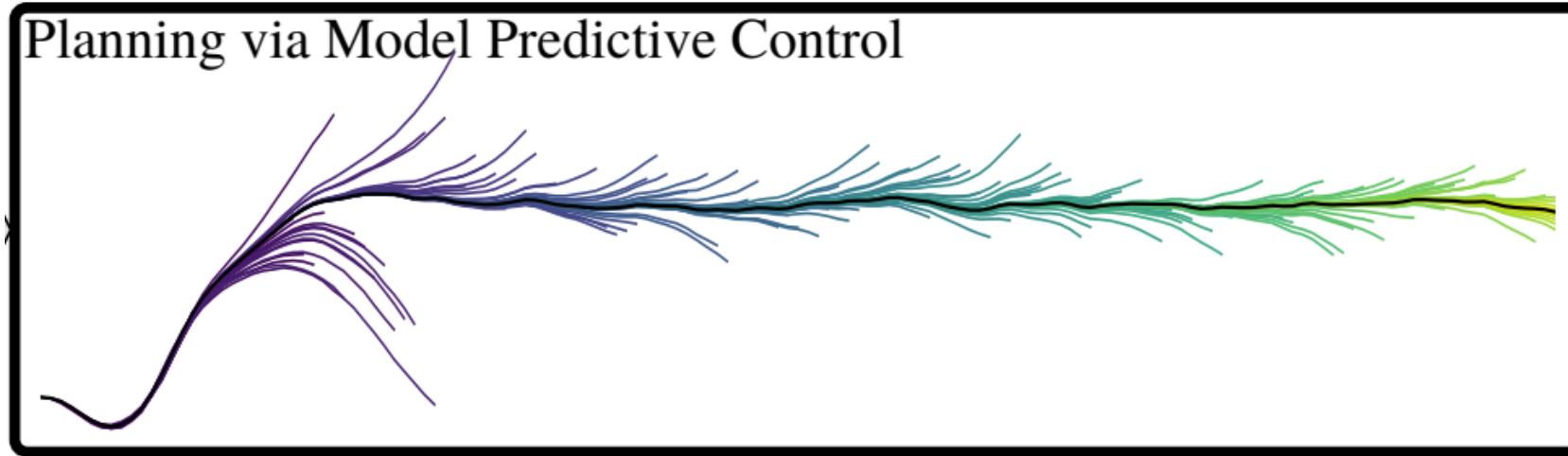
Probabilistic Neural Network

- Epistemic Uncertainty

- Subjective uncertainty due to lack of information (data)

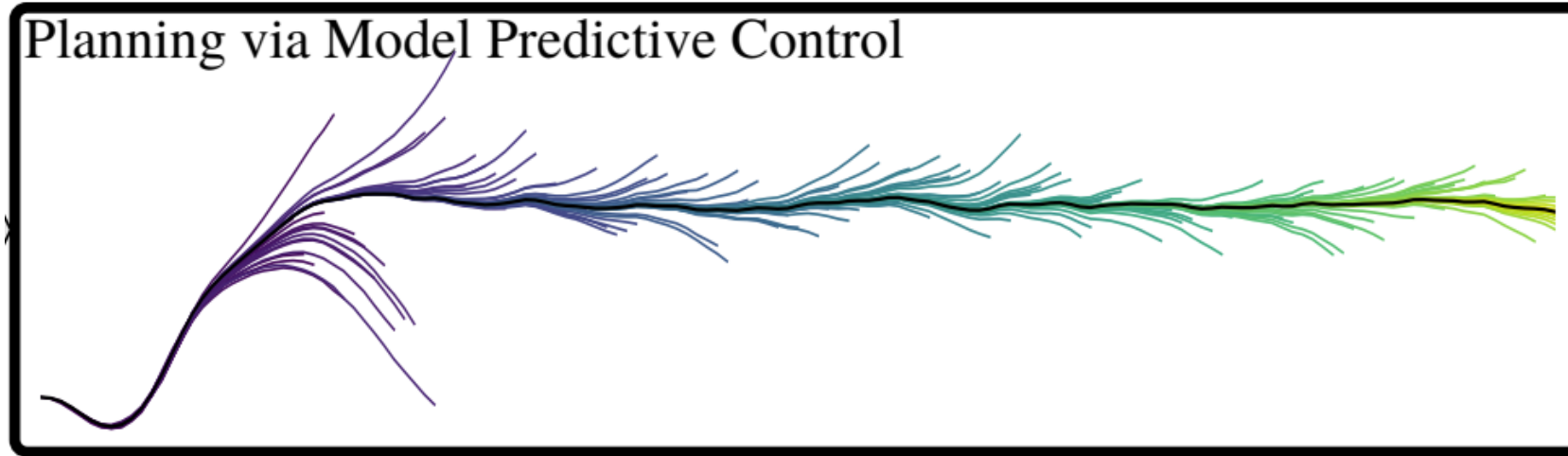
Bootstrapped Ensemble

How to use the model?



- Simplest version is **random-shooting**
 - Randomly sample actions sequences
 - Rollout each sequence using model
 - Evaluate based on predicted reward
 - Select best sequence and take first action
 - Repeat

How to use the model?



- Better version is the **Cross Entropy Method (CEM)** (evolutionary search)
 - Randomly sample actions sequences from a distribution (Gaussian)
 - Rollout each sequence using model
 - Evaluate based on predicted reward
 - Update distribution (Gaussian mean and variances) based on top-k sequences
 - Repeat
 - Select best sequence and take first action and repeat

But how do you do this with an ensemble?

- Similar to a particle filter
- Start with P particles (each representing state s_0)
- Each particle evolves according to a dynamics model from the ensemble to generate a simulated transition to a next state s_{t+1}
- Two ideas:
 - TS1: resample from ensemble each step
 - TS-infty: keep ensemble member fixed for full simulated rollout

Algorithm 1 Our model-based MPC algorithm ‘*PETS*’:

- 1: Initialize data $\mathbb{D}$ with a random controller for one trial.
 - 2: **for** Trial $k = 1$ to K **do**
 - 3: Train a *PE* dynamics model $\tilde{f}$ given $\mathbb{D}$.
 - 4: **for** Time $t = 0$ to TaskHorizon **do**
 - 5: **for** Actions sampled $\mathbf{a}_{t:t+T} \sim \text{CEM}(\cdot)$, 1 to NSamples **do**
 - 6: Propagate state particles $\mathbf{s}_\tau^p$ using *TS* and $\tilde{f}|\{\mathbb{D}, \mathbf{a}_{t:t+T}\}$.
 - 7: Evaluate actions as $\sum_{\tau=t}^{t+T} \frac{1}{P} \sum_{p=1}^P r(\mathbf{s}_\tau^p, \mathbf{a}_\tau)$
 - 8: Update $\text{CEM}(\cdot)$ distribution.
 - 9: Execute first action $\mathbf{a}_t^*$ (only) from optimal actions $\mathbf{a}_{t:t+T}^*$.
 - 10: Record outcome: $\mathbb{D} \leftarrow \mathbb{D} \cup \{\mathbf{s}_t, \mathbf{a}_t^*, \mathbf{s}_{t+1}\}$.
-